

Агент, способный изменять собственный код

Документация проекта

Санкт-Петербург, 2026

Содержание

1	Аннотация	3
2	Глоссарий	4
3	Постановка задачи	5
4	Эволюция архитектуры: V1, V2, V3, V4	6
4.1	V1 – оркестратор и кодер	6
4.2	V2 – одна модель на обе роли	6
4.3	V3 – память L0–L3	6
4.4	V4 – навыки	7
5	Mutable agent	8
5.1	Инструменты	8
5.2	Цикл ReAct	8
5.3	Протокол самомодификации	9
6	Библиотека llminterface	10
6.1	client – провайдер-независимый доступ к LLM	10
6.2	agent_chain – функциональная композиция	10
6.3	agent_graph – граф исполнения	11
7	Память L0–L3	12
8	Конституция агента	13
9	Тестирование моделей	14
9.1	Набор задач	14
9.2	Итоговый рейтинг: задачи оркестратора	15
9.3	Итоговый рейтинг: задачи кодера	15
9.4	Наблюдения	15
10	Модель данных	17
10.1	Цели функционирования	17
10.2	Описание сущностей	17
10.3	Лингвистическое описание	18
10.4	Атрибуты таблиц	18
10.5	Операции с данными	19
11	Брокер сообщений	21
11.1	Router RabbitMQ	21
11.2	Agent RabbitMQ	21
11.3	Класс RabbitMQBase	21
12	Контракты сообщений	23
12.1	Git-команды	23
12.2	GitRequest	23

13 Наблюдатель	24
13.1 Класс RabbitMQSupervisor	24
13.2 Обработчики сообщений	25
13.3 GitService	25
13.3.1 Назначение	25
13.3.2 Основные функции	25
13.3.3 Логика сохранения версии	26
13.4 Rollback	26
13.5 Логика работы наблюдателя	26
14 Маршрутизатор	28
14.1 Класс DockerManager	28
14.2 Класс RabbitMQRouter	29
14.3 RouterDatabase	29
14.4 Логика работы маршрутизатора	29
15 Протокол обмена сообщениями	31
15.1 Сообщения маршрутизатора	31
15.2 Сообщения наблюдателя	31
16 Docker-инфраструктура	32
16.1 Профиль router	32
16.2 Профиль agent	32
16.3 Тома и изоляция	32
17 Контроль качества и зависимости	33
18 Клиент	34
18.1 Структура клиента	34
18.2 Аутентификация и подключение	34
18.3 Работа с чатами	35
18.4 Рабочая директория пользователя	35
18.5 Выполнение клиентских команд	35
Заключение	37

1 Аннотация

В работе представлен агент, способный решать пользовательские задачи, использовать инструменты и изменять собственный код. Система разделена на изменяемую часть агента, неизменяемый модуль запуска и восстановления, супервизор, маршрутизатор, слой обмена сообщениями и хранилище данных. Такое разделение позволяет развивать поведение агента, не передавая ему контроль над критическими механизмами безопасности.

Основное внимание уделено управлению контекстом и памятью. В версии V3 контекст разделён на четыре уровня: постоянные правила L0, текущая задача L1, недавняя история L2 и сжатая долговременная память L3. Старые сообщения суммаризируются, поэтому объём промпта не растёт линейно вместе с длительностью диалога. История сообщений и L3 сохраняются в базе данных и могут быть восстановлены после перезапуска. В версии V4 добавлен механизм навыков, который сохраняет полезные последовательности действий после успешного выполнения и проверки задачи.

В документации описаны развитие архитектуры, конституция агента, механизм памяти, библиотека `llminterface`, на которой построен цикл рассуждений агента, а также результаты эмпирического тестирования агента на 15 языковых моделях. Отдельно рассматриваются саомодификация агента, работа инструментов, проверка завершённости задачи, автоматический `rollback` при ошибках, а также инфраструктурные компоненты: база данных, брокер сообщений, супервизор и маршрутизатор.

2 Глоссарий

Агент – программа на основе языковой модели, которая получает запрос, выбирает действия, вызывает инструменты и формирует ответ. **Mutable agent** – изменяемая часть системы в каталоге **agent/**, которую агент может дополнять в процессе работы. **Immutable agent** – неизменяемый модуль запуска, перезапуска и восстановления, который находится вне области самомодификации.

Supervisor (Наблюдатель) – отдельный компонент, контролирующий запуск агента, состояние версий и rollback после ошибки. **Router** (Маршрутизатор) – центральный оркестратор, отвечающий за аутентификацию пользователей и развёртывание изолированной инфраструктуры для каждой сессии. **Rollback** – возврат к последней работоспособной версии, если новая версия не запускается или завершает работу с ошибкой. **ReAct** – цикл работы модели, в котором рассуждение чередуется с вызовами инструментов и обработкой их результатов.

Tool – функция, доступная модели для выполнения конкретного действия, например чтения файла, запуска кода или поиска информации. **Target** – атрибут инструмента, определяющий, где именно он выполняется: на сервере (в контейнере агента) или на клиенте (на машине пользователя). **Skill** – сохранённое описание полезной последовательности действий, которое можно повторно использовать в похожих задачах. **Completion check** – дополнительная проверка того, что финальный ответ подтверждается результатами выполненных действий.

L0 – постоянная конституция и правила агента. **L1** – текущая задача пользователя. **L2** – недавняя часть диалога в пределах заданного лимита токенов. **L3** – сжатое резюме более старой истории. **LLMChat** – последовательность сообщений пользователя, модели и инструментов, которая сохраняется для восстановления контекста.

llminterface – внутренняя библиотека агента, предоставляющая провайдер-независимый доступ к языковым моделям (**client**), набор функциональных комбинаторов для композиции шагов обработки (**agent_chain**) и лёгкий движок графов состояний, на котором построен цикл ReAct (**agent_graph**). **Executable** – базовый интерфейс **agent_chain**, позволяющий объединять шаги обработки оператором **|**, аналогично цепочкам в **LangChain**. **AgentGraph** – граф исполнения с узлами, условными переходами и защитой от бесконечных циклов, применяемый для построения цикла рассуждений агента.

3 Постановка задачи

Необходимо разработать агента, который с помощью языковой модели выполняет пользовательские задачи и при необходимости дополняет собственный код. Изменение исходных файлов должно выполняться контролируемо: через команды операционной системы или полную перезапись файла. Код требуется разделять на небольшие модули, чтобы отдельные уровни системы можно было развивать и проверять независимо.

Архитектура должна разделять взаимодействие с LLM, доступ к данным и работу с операционной системой. Конституция и личность агента хранятся отдельно и не могут изменяться самим агентом. Изменяемый код размещается в `agent/`, пользовательские результаты создаются непосредственно на машине пользователя, а критические каталоги подключаются к контейнеру без права записи.

Перед применением новой версии должен сохраняться полный снимок предыдущего состояния. Модуль перезапуска и логика rollback работают отдельно от изменяемого агента и не могут быть переписаны им. Если новая версия завершается с ошибкой, система фиксирует причину, восстанавливает последнюю стабильную версию. Частичное редактирование через `sed` не используется, основной способ изменения исходников – полная запись нового содержимого файла.

Помимо саомодификации, агент должен поддерживать продолжительный диалог, безопасно разделять действия, выполняемые на сервере и на машине пользователя, и позволять сравнивать качество своей работы на разных языковых моделях, чтобы обоснованно выбирать модель по умолчанию.

4 Эволюция архитектуры: V1, V2, V3, V4

В ходе практики архитектура агента менялась поэтапно. В каждой версии мы старались уменьшить расход токенов и число лишних вызовов модели, сохраняя контроль над само-модификацией. Ниже описаны основные версии и причины изменений.

4.1 V1 – оркестратор и кодер

В первой версии использовались две модели с разными ролями. Оркестратор планировал шаги и выбирал инструменты, а кодер генерировал и изменял код. Передача контекста между ними увеличивала число вызовов и общий расход токенов. Для типовой задачи с обращением к публичному API и построением результата в V1 были получены ориентировочные метрики порядка 138 секунд, 10 вызовов модели и около 19 800 токенов.

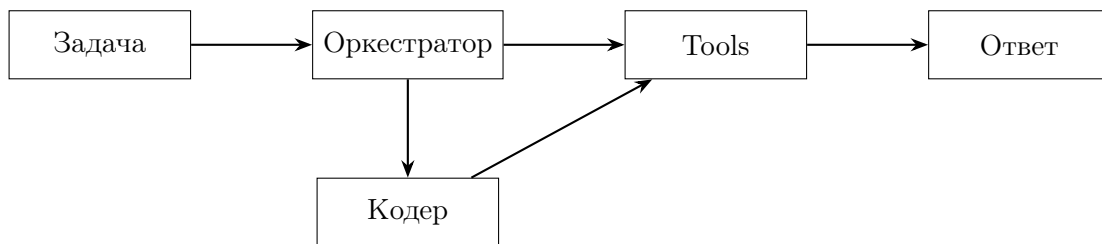


Рис. 1: Упрощённая схема V1: две модели и общий контур инструментов

4.2 V2 – одна модель на обе роли

Во второй версии планирование и написание кода были объединены в одной модели. Архитектура стала проще: исчезло постоянное переключение между оркестратором и кодером, уменьшилось число служебных вызовов. При этом основная проблема осталась: на каждом следующем шаге в промпт снова попадала почти вся история диалога. Поэтому выигрыш V2 относительно V1 был заметным, но умеренным. На той же задаче ориентировочно время сократилось примерно до 125 секунд, число вызовов – до 8, а токены – до около 17 600.

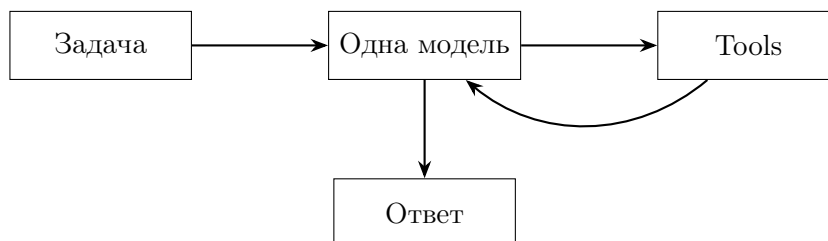


Рис. 2: Упрощённая схема V2: единая модель в цикле ReAct

4.3 V3 – память L0–L3

Третья версия сохранила одну модель, но изменила способ сборки контекста. Вместо того чтобы каждый раз отправлять весь диалог дословно, контекст был разделён на уровни. Конституция и правила образуют постоянный слой L0, текущая задача – слой L1, недавние шаги – рабочий хвост L2, а старые шаги сжимаются в краткие резюме L3. Благодаря этому промпт перестаёт расти линейно с длиной сессии. Именно на длинных многошаговых задачах преимущество V3 становится наиболее заметным: свежие шаги остаются подробными, а старые передаются кратко.

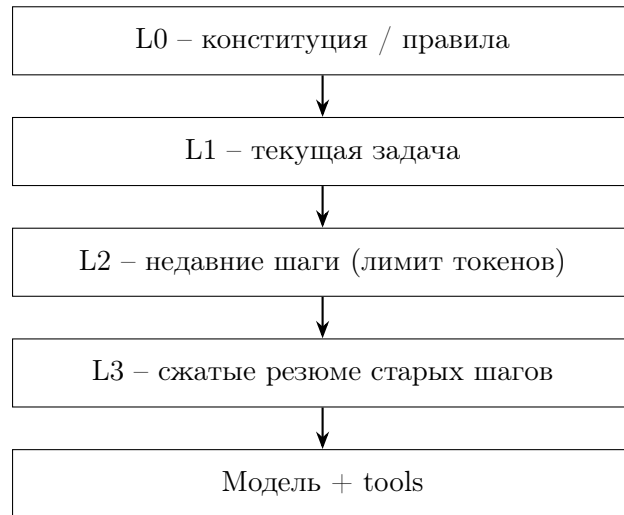


Рис. 3: Упрощённая схема V3: слоистый контекст перед вызовом модели

4.4 V4 – навыки

В четвёртой версии к архитектуре V3 добавлен механизм навыков. После успешного выполнения задачи отдельная проверка сначала подтверждает, что результат действительно получен. Только после этого система анализирует выполненную последовательность действий. Если агент сделал не менее трёх вызовов инструментов и использовал не менее двух разных инструментов, суммаризатор определяет, содержит ли задача новую переиспользуемую процедуру.

Если такая процедура найдена, она сохраняется в отдельном Markdown-файле как skill. В файл входят краткое описание, список использованных инструментов и последовательность шагов. Пользовательские данные, секреты, точные имена файлов и одноразовые детали в навык не переносятся. За одну задачу создаётся не более одного навыка, а незавершённая или не прошедшая проверку задача не может стать источником skill. Сохранённый навык можно использовать в следующих похожих задачах без добавления полной истории предыдущей работы в контекст.

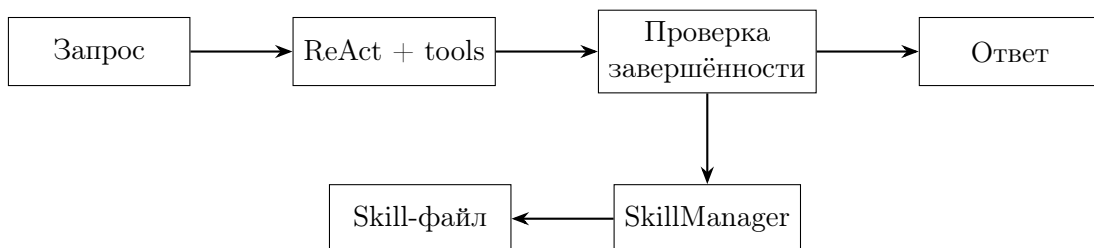


Рис. 4: Упрощённая схема V4: создание навыка после проверки задачи

5 Mutable agent

Mutable agent – это единственная часть системы, которую агент может изменять во время своей работы. Она находится в каталоге `agent/` и включает цикл рассуждений, набор инструментов, работу с памятью, проверку завершённости задачи и механизм навыков. Всё, что выходит за пределы этого каталога (неизменяемый модуль запуска, конституция, супервизор), агенту недоступно на запись.

5.1 Инструменты

Инструменты агента регистрируются декоратором `@tool` из `agent/tools/registry.py`. Декоратор не требует ручного описания JSON-схемы: сигнатура функции и её docstring разбираются автоматически (краткое описание, описание параметров в формате Google-или Sphinx-стиля), а обязательность параметра определяется наличием значения по умолчанию. Новый файл с единственным инструментом в `agent/tools/` подхватывается автоматически при старте процесса через `pkgutil.iter_modules` – зарегистрировать инструмент вручную в `__init__.py` не требуется.

Ключевая особенность модели инструментов – разделение на две стороны выполнения: *сервер* (контейнер агента) и *клиент* (машина пользователя, подключённая через RabbitMQ-мост). У каждого инструмента заранее зафиксирован допустимый набор сторон (`allowed_targets`) и сторона по умолчанию (`default_target`):

- Подавляющее большинство инструментов (файлы, веб-поиск, git-версии, создание новых инструментов) жёстко закреплены за сервером – их схема не содержит параметра `target`, и модель не может перенаправить их на клиент.
- `ask_user` жёстко закреплён за клиентом – он должен достигаться до реального человека, а не до контейнера агента.
- `run_bash`, `run_powershell` и `run_python` – единственные инструменты, для которых сторона выбирается моделью на каждый вызов (по умолчанию – клиент). Это осознанное архитектурное решение: сервер и клиент физически являются разными машинами с разными файловыми системами, и одна и та же команда, выполненная не на той стороне, не увидит нужных файлов и не решит задачу пользователя.

Маршрутизация вызова определяется функцией `tool_target`: она берёт фиксированную сторону инструмента и, только если инструмент допускает выбор, уважает явный параметр `target` из аргументов вызова. Вызов с `target='server'` выполняется напрямую в процессе агента (`execute_tool`); вызов с `target='client'` превращается в сообщение типа `client_command`, уходит через супервизора клиенту по RPC-паттерну (`reply_to / correlation_id`) и ждёт ответа с таймаутом. Отдельно для `run_bash/run_powershell` на стороне сервера запрещено выполнение команд, содержащих слово `git`: рабочий каталог агента уже находится под управлением собственной git-версионности супервизора, и прямой вызов `git` в обход `version_commit/version_rollback` способен рассинхронизировать историю снимков.

5.2 Цикл ReAct

Цикл рассуждений построен как граф на движке `agent_graph` из библиотеки `llminterface` (см. следующий раздел). Основные узлы:

- **start** – фиксирует новую задачу в памяти и на первом шаге сессии добавляет в контекст каталог сохранённых навыков (`skills_list`), чтобы модель не начинала знакомую задачу с нуля.
- **model** – собирает промпт из уровней памяти L0–L3, вызывает языковую модель потоково и записывает её ответ (включая запрошенные вызовы инструментов) в память.
- **tools** – выполняет запрошенные вызовы параллельно, отслеживает подряд идущие неудачные попытки одного и того же инструмента и подсказывает модели свериться с каталогом навыков после нескольких провалов подряд.
- **review** – проверяет черновой финальный ответ через `CompletionChecker`: подтверждается ли ответ реальными результатами инструментов, а не только заявлением модели об успехе.
- **finalize** – принимает подтверждённый ответ, передаёт его пользователю и запускает `SkillManager`, который решает, стоит ли сохранить пройденную последовательность действий как новый навык.
- **verification_failed** и **limit** – терминальные узлы для случая, когда проверка так и не прошла (после ограниченного числа попыток) или закончился лимит итераций.

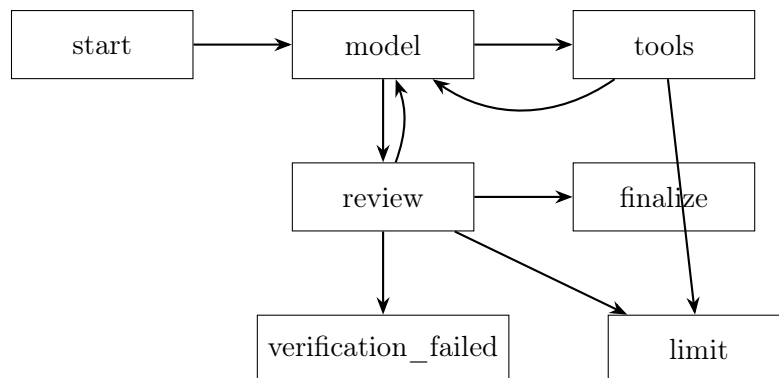


Рис. 5: Граф цикла ReAct: `model` и `tools` образуют внутренний цикл, `review` принимает решение о завершении задачи

5.3 Протокол самомодификации

Изменение собственного кода подчинено простому протоколу: прочитать целевой модуль (`read_file`), полностью переписать его новым содержимым (`write_file`) и зафиксировать изменение инструментом `version_commit`. Частичные правки (`sed`, `insert_text`) для исходников агента запрещены конституцией – незавершённое изменение может нарушить синтаксис и сломать агент при следующем перезапуске. Именно `version_commit` запускает проверку синтаксиса изменённых модулей на стороне супервизора и, при успехе, перезапуск контейнера агента: новая логика становится действующей только после этого перезапуска, а не сразу после записи файла.

6 Библиотека llminterface

`agent/llminterface/` – это внутренняя, не зависящая от внешних agent-фреймворков библиотека, на которой построен весь цикл ReAct. Она сознательно минималистична: вместо тяжёлых зависимостей вроде LangChain или LangGraph агент использует три небольших, независимо тестируемых модуля – `client` (доступ к LLM), `agent_chain` (функциональная композиция шагов) и `agent_graph` (граф исполнения с состоянием).

6.1 client – провайдер-независимый доступ к LLM

Модуль `client` отделяет формат общения с конкретным провайдером от остальной системы. Абстрактный класс `LLMClient` задаёт единый интерфейс: методы `send` (один запрос-ответ) и `stream` (потокковая генерация с колбэками на каждый фрагмент рассуждения и содержимого ответа). Каждый провайдер реализует только приватный метод `_parse_response`, который переводит сырой ответ API в провайдер-агностичную модель данных – только в этом методе клиент вообще "знает" формат своего API.

Провайдер-агностичная модель сообщений (`LLMChat`, `LLMMessage`) – отдельный слой, не привязанный ни к одному конкретному API. `LLMMessage` хранит роль, содержимое ответа отдельно от содержимого рассуждения (`content` и `thinking` – важно для reasoning-моделей, которые иногда возвращают весь ответ только в поле рассуждения), запрошенные вызовы инструментов, а также метрики – токены (`LLMTokens`) и длительности этапов (`LLMDuration`: загрузка модели, обработка промпта, генерация ответа). `LLMChat` – список сообщений с перегруженными операторами сложения (`chat + message`), что позволяет строить историю диалога декларативно, без явных вызовов `.append()`.

В проекте реализовано два провайдера:

- `OllamaClient` – работает с локальным или удалённым Ollama-сервером по протоколу `/api/chat`, поддерживает потоковый и непотоковый режим.
- `OpenaiClient` – работает с любым OpenAI-совместимым API (сам OpenAI, а также OpenRouter и подобные шлюзы, различающиеся только базовым URL и ключом).

Оба провайдера взаимозаменяемы с точки зрения остального агента: цикл ReAct обращается только к абстракции `LLMClient` и никогда не знает, с каким именно провайдером сейчас работает.

6.2 agent_chain – функциональная композиция

Модуль `agent_chain` задаёт единый интерфейс `Executable` с методами `run` и `stream`, а также перегруженный оператор `|`, позволяющий собирать шаги обработки в цепочку декларативно – по аналогии с LCEL в LangChain, но без внешней зависимости. Библиотека `execs.py` предоставляет набор готовых комбинаторов:

- `ExecSequence` – последовательное исполнение шагов, результат одного передаётся следующему;
- `ExecParallel` – параллельный запуск нескольких ветвей с объединением результатов в словарь;
- `ExecLambda` – превращает произвольную функцию в `Executable`;
- `ExecEffect` – выполняет побочный эффект (например, отправку события в UI), но пропускает исходные данные дальше без изменений;

- `ExecUpdate` – берёт объект с методом `to_dict()` (в частности, `AgentState`) и возвращает копию с обновлёнными полями;
- набор более специализированных комбинаторов: `ExecPassthrough`, `ExecPartial`, `ExecMultiargument`, `ExecSelect`, `ExecCall`.

На практике это позволяет описывать узел графа одним выражением. Например, узел `model` цикла `ReAct` (см. предыдущий раздел) собран именно так:

```
ExecLambda(prepare_context)
| ExecEffect(...)
| ExecUpdate(chat=ExecLambda(call_model))
| ExecEffect(record_assistant)
```

– собрать контекст, известить UI о начале ответа модели, вызвать модель и обновить состояние её ответом, записать ответ в память. Каждый шаг тестируется независимо от остальных, а сама последовательность остаётся читаемой в одну строку.

Класс `LLMClient` тоже наследуется от `Executable` (его `run` делегирует в `send`) – это позволяет использовать сам вызов модели как обычный узел цепочки наравне с любым другим шагом.

6.3 `agent_graph` – граф исполнения

Модуль `agent_graph` предоставляет лёгкий движок графов состояний: `AgentState` – изменяемое состояние с точечными обновлениями и опциональными редьюсерами (например, накопление в список вместо перезаписи поля), и `AgentGraph` – сам граф с узлами (`add_node`), безусловными и условными переходами (`add_edge`, `add_conditional_edge`) и одной или несколькими точками входа.

Каждый узел графа приводится к `Executable` автоматически (обычная функция обрачивается в `ExecLambda`, словарь – в `ExecParallel`), поэтому узлом графа может быть как простая функция, так и цепочка, собранная через `agent_chain`. Обход графа идёт "по фронту": на каждом шаге исполняются все узлы текущего фронта (параллельно через пул потоков, если их больше одного), их обновления сливаются в общее состояние, а следующий фронт определяется явными рёбрами или условным роутером. Встроенная защита `max_steps` останавливает исполнение, если граф заиклился.

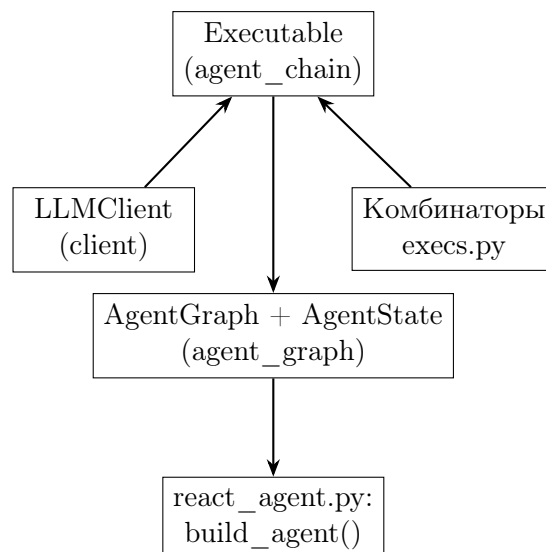


Рис. 6: Слои `llminterface`: `LLMClient` и комбинаторы реализуют `Executable`, граф исполняет составленные из них узлы

7 Память L0–L3

Память агента в версии V3 устроена как набор уровней, из которых перед каждым вызовом модели собирается рабочий промпт. Уровень L0 содержит конституцию и постоянные правила. Уровень L1 хранит текущий пользовательский запрос и цель сессии. Уровень L2 – это недавний хвост диалога: свежие шаги, ответы модели и результаты инструментов. Уровень L3 хранит сжатые резюме более старых шагов, которые уже невыгодно держать в промпте дословно.

Размер L2 ограничивается не числом сообщений, а лимитом токенов. Сообщения сильно различаются по объёму: короткая реплика почти ничего не стоит, а вывод чтения большого файла может занимать тысячи токенов. Поэтому в коде задаётся фиксированный лимит, например несколько тысяч токенов. Пока свежий хвост не превышает этот лимит, он остаётся в L2. Когда лимит превышен, самые старые шаги уходят на суммаризацию, результат добавляется в L3, а из рабочего хвоста эти шаги удаляются. Полный лог при этом может сохраняться на диске отдельно, чтобы история не терялась для отладки.

Такой подход уменьшает размер промпта, число токенов, время ответа и стоимость длинных сессий. В версиях V1 и V2 контекст рос почти линейно: на каждом следующем шаге модель снова получала всю прошлую историю. В V3 свежие сообщения остаются подробными, а старые передаются кратко. На короткой одноходовой задаче разница может быть небольшой, но она растёт по мере продолжения диалога.

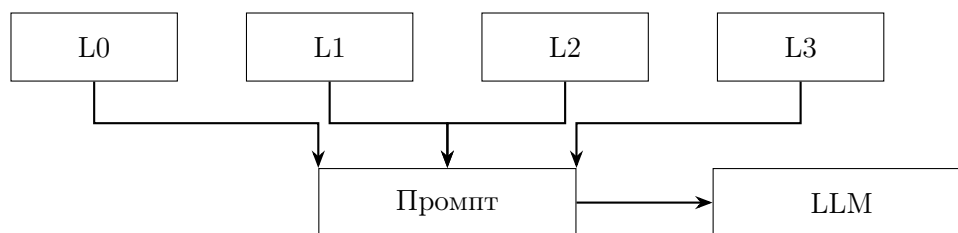


Рис. 7: Сборка промпта из уровней памяти L0–L3

8 Конституция агента

Конституция – это набор правил и описание личности агента, которые задают допустимое поведение до начала решения конкретной задачи. В проекте она хранится отдельно от изменяемого кода, в файле `constitution/identity.md`, и рассматривается как слой памяти L0. Именно конституция фиксирует, на каком языке отвечать, какие инструменты предпочитать, куда разрешено писать файлы и какие каталоги трогать нельзя.

С точки зрения архитектуры конституция намеренно вынесена за пределы `mutable agent`. Агент может изменять собственный код в `agent/`, но не должен переписывать правила, по которым он работает. Если бы агент мог свободно менять конституцию, он смог бы снять с себя ограничения: разрешить запись в критические директории, отключить требования к проверкам или изменить протокол самомодификации. Поэтому допустимые области записи зафиксированы в конституции, а критические каталоги, включая `constitution/` и `agent_immutable/`, в Docker-контуре монтируются как `read-only`.

Конституция состоит из нескольких смысловых блоков:

- **Права на запись** – агенту разрешено писать только в `agent/` (собственный код) и в собственный рабочий каталог на сервере, предназначенный для черновой работы. Изменение собственного кода допускается только полной перезаписью файла (`write_file`), частичные правки инструментом `insert_text` для исходников агента запрещены – незавершённое изменение может сломать агент при следующем перезапуске.
- **Где выполняются инструменты** – явно перечислено, какие инструменты вообще способны выполняться на машине пользователя (их ровно четыре: `ask_user`, `run_bash`, `run_powershell`, `run_python`), а какие жёстко привязаны к серверу и не принимают параметр `target` вовсе. Отдельно оговорено, что то же разделение действует и для сетевых запросов: запрос, отправленный серверным инструментом, отражает сетевую принадлежность контейнера агента, а не пользователя.
- **Результаты для пользователя** – любой файл, который пользователь должен открыть на своей машине, обязан создаваться инструментом с `target='client'`; между сервером и клиентом нет инструмента передачи файлов, поэтому вместо поиска способа переслать уже созданный на сервере файл нужно повторить его построение на стороне клиента.
- **Протокол самомодификации и добавление новых инструментов** – порядок действий при изменении своего кода и правило, что новый инструмент всегда оформляется отдельным файлом в `agent/tools/`, обнаруживаемым автоматически.
- **Работа с незнакомыми библиотеками** – перед интеграцией с внешней библиотекой предписано сначала свериться с каталогом сохранённых навыков (`skills_list` / `skill_view`), а при повторяющихся ошибках – переходить к интроспекции установленной версии библиотеки, а не к угадыванию API.

Конституция читается на каждой задаче заново вместе с отдельным блоком *runtime guidance* (актуальная дата, напоминание о выборе `target` для результатов пользователя, инструкция по каталогу навыков) – это позволяет держать самые важные, часто нарушаемые правила не только в статичном L0, но и в динамически формируемой, более «свежей» части промпта.

9 Тестирование моделей

Поскольку архитектура агента не завязана на конкретного провайдера LLM (см. раздел 6), было проведено эмпирическое сравнение 15 моделей, доступных через Ollama, на одном и том же наборе задач и одной и той же версии агента. Всего выполнено $15 \times 11 = 165$ прогонов: 8 задач в роли “оркестратора” (работа с файлами, интернетом, условной логикой – то, что решает сам агент в проекте) и 3 задачи в роли “кодера” (написание и редактирование кода без выполнения). Оценка каждого прогона (от 1 до 5) выставлялась вручную по истории сообщений и итоговому ответу или файлам; время выполнения и число токенов фиксировались отдельно и не влияли на оценку. Средний балл по всем 165 прогонам составил 4.00 из 5.

9.1 Набор задач

- O1. Сумма столбца в CSV – простая (1–2 вызова инструментов).
- O2. Сверка версии Python с официальным сайтом – тяжёлая (веб-поиск + bash + сравнение + запись файла).
- O3. Обновление зависимости в `config.json` – средняя (редактирование структурированного файла + интернет).
- O4. Актуальная версия pipru через интернет – лёгкая (интернет + структурированный файл).
- O5. Погода в текущем городе пользователя – средняя-тяжёлая (определение геолокации + веб-поиск данных о погоде).
- O6. Агрегация чисел из нескольких файлов – средняя (несколько однотипных вызовов + агрегация).
- O7. Лучшие игры до 300 рублей на летней распродаже Steam – тяжёлая (веб-поиск + агрегация + субъективный отбор).
- O8. Аудит уязвимостей зависимостей (отчёт в Markdown и Excel, условное предупреждение) – тяжёлая (несколько независимых веток поиска + агрегация + условная логика).
- K1. Проверка строки на палиндром – простая (написание с нуля).
- K2. Распараллеливание загрузки URL через `ThreadPoolExecutor` – редактирование существующего кода.
- K3. LRU-кэш с ручной реализацией двусвязного списка – тяжёлая (алгоритмическое проектирование с нуля).

Задачи O5 (геолокация пользователя) и O7 (подбор товаров по актуальной цене) намеренно требуют актуальных данных из интернета и не имеют единственно верного ответа, что позволяет проверить не только техническую точность, но и добросовестность модели – пользуется ли она инструментами для получения фактов или додумывает правдоподобный ответ.

9.2 Итоговый рейтинг: задачи оркестратора

#	Модель	Балл
1	ornith:35b	4.50
2	qwen3.6-35b:apex-iquality	4.38
3	qwen3.6-35b:mtp-q4km	4.38
4	glm-4.7-flash:q4_K_M	4.38
5	ornith:9b	4.25
6	qwen3.6:35b	4.12
7	gemma4:12b	4.12
8	qwythos-9b:mtp-q4km	4.12
9	qwen3.6-35b:lordx64-opus-distill	4.00
10	gemma4:26b	4.00
11	qwen3.5:9b-32k	3.50
12	gemma4:latest	3.38
13	gpt-oss:20b	3.25
14	nemotron-cascade-2:latest	2.75
15	gemma4:e2b	2.75

9.3 Итоговый рейтинг: задачи кодера

#	Модель	Балл
1	ornith:35b	5.00
2	qwen3.6-35b:apex-iquality	4.67
3	qwen3.6-35b:lordx64-opus-distill	4.67
4	gemma4:26b	4.67
5	gemma4:latest	4.67
6	gpt-oss:20b	4.67
7	nemotron-cascade-2:latest	4.67
8	qwen3.6-35b:mtp-q4km	4.33
9	qwen3.6:35b	4.33
10	gemma4:12b	4.33
11	qwythos-9b:mtp-q4km	4.33
12	gemma4:e2b	4.33
13	glm-4.7-flash:q4_K_M	4.00
14	ornith:9b	3.67
15	qwen3.5:9b-32k	3.33

Заметно, что рейтинг по задачам кодера заметно отличается от рейтинга по задачам оркестратора: например, qwen3.6-35b:lordx64-opus-distill и nemotron-cascade-2:latest поднимаются с середины списка на второе место, а glm-4.7-flash:q4_K_M, наоборот, опускается – сказывается обнаруженный в задаче K2 функциональный баг (не проверяет HTTP-статус ответа при параллельной загрузке).

9.4 Наблюдения

Несколько закономерностей повторяются независимо от конкретной модели. Во-первых, самая тяжёлая задача оркестратора – аудит уязвимостей зависимостей с экспортом в Excel

– систематически проваливалась или сильно замедляла даже сильные модели из-за одной и той же технической ловушки при работе с `openruhl`; это разошлось по нескольким моделям независимо, то есть указывает на реальный пробел в обучающих данных моделей для этой конкретной библиотеки, а не на случайность. Во-вторых, задачи с открытым, актуальным ответом (погода, распродажа Steam) хорошо различают модели, которые добросовестно используют инструменты, от моделей, которые фабрикуют правдоподобные, но неверные данные – этот разрыв не виден на задачах с детерминированным ответом. В-третьих, компактные модели (9B) на простых задачах нередко обгоняют по времени и даже по качеству более крупные (35B) модели, но проигрывают на задачах, требующих длинного исследования в несколько независимых веток.

Лучший результат бенчмарка – `ornith:35b`, с большим отрывом первая и на задачах оркестратора, и на задачах кодера; это значение использовалось как ориентир при выборе модели по умолчанию для агента.

10 Модель данных

Для хранения информации о снимках рабочей директории агента, ошибках и содержании LLM-чата внутри docker-контейнера `hyperagent_postgres` разворачивается база данных Postgres. Для нее используется образ `postgres:15-alpine`.

10.1 Цели функционирования

1. Учет всех снимков рабочей директории агента в ходе его работы.
2. Учет всех ошибок, которые возникают в результате изменения агентом своего кода.
3. Учет всех сообщений от LLM-модели в ходе работы агента.
4. Хранение пользовательских чатов и клиентских сообщений.
5. Хранение L3 памяти.

10.2 Описание сущностей

1. Снимок - набор сведений, характеризующий состояние рабочей директории агента, полученное в результате ее изменения. Содержит всю необходимую информацию для выполнения отката к этому состоянию. Также содержит информацию о статусе конкретного состояния: стабильное, нестабильное или ошибочное.
Атрибуты: ссылка на git-коммит, статус, время создания, описание изменения.
2. Ошибка - событие, которое происходит, если агент вносит в свой код изменения, которые могут привести к нарушению его функционирования.
Атрибуты: текст ошибки, снимок, время возникновения.
3. Чат - отдельная пользовательская сессия взаимодействия с системой.
Атрибуты: идентификатор, название, время последнего обновления.
4. Клиентское сообщение - сообщение, отправленное пользователем или клиентским приложением в рамках конкретного чата.
Атрибуты: идентификатор, чат, тип сообщения, содержимое сообщения, время создания.
5. LLM-сообщение - набор сведений, который характеризует сообщение от LLM-модели.
Атрибуты: чат, завершение, причина завершения, роль, размышление, ответ, вызовы инструментов, идентификатор вызова, имя инструмента, поставщик, модель, токены для восстановления контекста, токены для генерации, время загрузки, время восстановления контекста, время генерации, время сообщения.
6. L3 Memory - долговременное сжатое представление истории чата. Используется для хранения краткого содержания диалога, которое сможет использовать LLM.
Атрибуты: чат, краткое содержание, время создания, идентификатор последнего учтенного сообщения.

10.3 Лингвистическое описание

1. Snapshot(<id, serial>, <sha, varchar(40)>, <status, varchar(20)>, <snapshot_time, timestamp>, <modification, text>)
2. Error(<id, serial>, <snapshot_id, int4>, <error_text, text>, <error_time, timestamp>)
3. Chat(<id, serial>, <title, text>, <updated_at, timestamp>)
4. ClientChat(<id, serial>, <chat_id, int>, <message_type, text>, <message, jsonb>, <dt, timestamp>)
5. LLMChat(<id, serial>, <chat_id, int>, <done, boolean>, <done_reason, text>, <role, varchar(10)>, <thinking, text>, <content, text>, <tool_calls, jsonb>, <tool_call_id, text>, <name, text>, <provider, text>, <model, text>, <tokens_prompt, int>, <tokens_response, int>, <duration_load, int>, <duration_prompt, int>, <duration_response, int>, <dt, timestamp>)
6. L3Memory(<chat_id, int>, <summary, text>, <created_at, timestamp>, <last_message_id, int>)

10.4 Атрибуты таблиц

Таблица . Snapshot.

Атрибут	Тип	Назначение
id	serial	Первичный ключ снимка.
sha	varchar(40)	Ссылка на git-коммит.
status	varchar(20)	Статус снимка: PENDING, STABLE или ERROR.
snapshot_time	timestamp	Время создания снимка.
modification	text	Описание изменения или сообщение коммита.

Таблица . Error.

Атрибут	Тип	Назначение
id	serial	Первичный ключ ошибки.
snapshot_id	int4	Ссылка на снимок, с которым связана ошибка.
error_text	text	Текст ошибки.
error_time	timestamp	Время регистрации ошибки.

Таблица . Chat.

Атрибут	Тип	Назначение
id	serial	Первичный ключ чата.
title	text	Название чата.
updated_at	timestampz	Время последнего обновления чата.

Таблица . ClientChat.

Атрибут	Тип	Назначение
id	serial	Первичный ключ клиентского сообщения.
chat_id	int	Ссылка на чат.
message_type	text	Тип клиентского сообщения.
message	jsonb	Содержимое сообщения в формате JSON.
dt	timestampz	Время создания сообщения.

Таблица . LLMChat.

Атрибут	Тип	Назначение
id	serial	Первичный ключ сообщения.
chat_id	int	Ссылка на чат.
done	boolean	Признак завершенности ответа.
done_reason	text	Причина завершения ответа.
role	varchar(10)	Роль сообщения: assistant, system, user или tool.
thinking	text	Размышление модели.
content	text	Основное содержимое сообщения.
tool_calls	jsonb	Вызовы инструментов.
tool_call_id	text	Идентификатор вызова инструмента.
name	text	Имя инструмента или сообщения.
provider	text	Поставщик модели.
model	text	Название модели.
tokens_prompt	int	Количество токенов входного контекста.
tokens_response	int	Количество токенов ответа.
duration_load	int	Время загрузки.
duration_prompt	int	Время обработки контекста.
duration_response	int	Время генерации ответа.
dt	timestampz	Время сообщения.

Таблица . L3Memory.

Атрибут	Тип	Назначение
chat_id	int	Первичный ключ и ссылка на чат.
summary	text	Сжатое описание истории чата.
created_at	timestamp	Время создания или обновления памяти.
last_message_id	int	Последнее сообщение, учтенное при сжатии.

10.5 Операции с данными

Для взаимодействия с базами данных PostgreSQL используется SQLAlchemy версии 2.0.36 в качестве ORM. Операции CRUD разделены на два независимых модуля, соответствующих микросервисам:

- Supervisor CRUD - управление снимками кода Snapshot и ошибками Error.
- Agent CRUD - управление историей диалогов, чатами и долговременной памятью.
- Router CRUD - управление учетными записями пользователей маршрутизатора.

Функции Supervisor CRUD:

- add_snapshot - создание новой записи снимка.
- get_snapshot_by_status - получение самого последнего снимка указанного статуса.
- get_stable_snapshots - получение списка всех стабильных снимков для отображения истории коммитов.
- update_snapshot_status - изменение статуса существующего снимка.
- add_error - регистрация ошибки, связанной с конкретным снимком.
- get_errors - получение всех ошибок для конкретного снимка.

Функции Agent CRUD:

- list_chats - получение списка всех чатов пользователя.
- create_chat - создание нового чата.
- rename_chat - изменение названия существующего чата.
- get_chat_history - получение истории клиентских сообщений конкретного чата.
- add_client_message - сохранение нового клиентского сообщения.
- add_message - сохранение нового сообщения от пользователя, агента или инструмента в историю LLM-чата.
- get_llmchat - загрузка полной истории LLM-сообщений для конкретного чата.
- get_l3_memory - получение долговременной памяти чата.
- add_l3_memory - создание или обновление долговременной памяти чата.

Функции Router CRUD:

- add_user - создание записи пользователя, сохранение хеша пароля и выделенных портов.
- get_user - получение пользователя по логину.

11 Брокер сообщений

Для надежной доставки сообщений между распределенными компонентами системы используется брокер сообщений RabbitMQ.

В системе используется два основных обменника и несколько очередей, маршрутизация между которыми осуществляется через ключи маршрутизации.

- Router Exchange - используется для взаимодействия внешнего клиента с центральным маршрутизатором. Центральный маршрутизатор имеет свою очередь, в которую можно писать по ключу router.
- Agent Exchange - используется для внутренней коммуникации между наблюдателем, агентом и клиентом после успешной аутентификации. Для каждого из них выделяется отдельная очередь с ключом маршрутизации, соответствующим названию модуля.

11.1 Router RabbitMQ

Брокер маршрутизатора содержит пользователя router и пользователя client. Клиент отправляет login и logout-запросы в обменник router_exchange. Сообщения с routing key router попадают в очередь router_queue, которую слушает маршрутизатор.

Для ответа клиенту используется RPC-подход: клиент создает временную очередь ответа, указывает ее в поле reply_to, а маршрутизатор публикует ответ в exchange по умолчанию с сохранением correlation_id.

11.2 Agent RabbitMQ

Персональный брокер пользователя содержит пользователей agent, supervisor и client. В нем создается обменник agent_exchange и три основные очереди:

- agent_queue - очередь агента, routing key agent.
- supervisor_queue - очередь наблюдателя, routing key supervisor.
- client_queue - очередь клиента, routing key client.

Такая структура позволяет разделить сообщения по получателям и не смешивать команды управления, ответы агента и уведомления клиента.

11.3 Класс RabbitMQBase

RabbitMQBase - это базовый класс, инкапсулирующий низкоуровневую логику взаимодействия с брокером сообщений RabbitMQ с использованием библиотеки pika версии 1.3.2. В конструкторе класса устанавливается блокирующее соединение с RabbitMQ connection, настраивается канал связи channel для выполнения операций.

Класс содержит следующие методы:

- Метод publish_message. Публикация сообщения в обменник. Вызывает channel.basic_publish, указывая exchange, routing_key и body. Устанавливает delivery_mode=2, чтобы сообщение сохранялось на диск брокера и не терялось при его перезагрузке.

- Метод `start_consuming`. Запуск бесконечного цикла ожидания и обработки входящих сообщений. Привязывает метод `receive_message` как `callback`-функцию к очереди. Вызывает `channel.start_consuming()`, блокируя текущий поток.
- Метод `request_response`. Реализация RPC-запроса. Создает или переиспользует временную очередь ответа, отправляет сообщение с `correlation_id` и ожидает ответ в течение заданного `timeout`.
- Метод `send_response`. Отправка ответа на RPC-запрос в очередь, указанную в `reply_to`.
- Метод `receive_message`. Абстрактный метод обработки входящего сообщения. Должен быть реализован в наследниках.

В случае отсутствия ответа в течение заданного времени используется исключение `RequestResponseTimeoutError`. Оно позволяет отличать сетевые ошибки от ситуации, когда получатель не вернул ответ на RPC-запрос.

12 Контракты сообщений

Для строгого описания сообщений, которые передаются между компонентами системы, используется отдельный модуль `contracts`. Он содержит Pydantic-модели, задающие допустимые типы команд и структуру данных для запросов.

12.1 Git-команды

Git-команды используются для взаимодействия агента с наблюдателем и позволяют получить состояние репозитория, посмотреть изменения, сохранить новую версию кода или выполнить откат.

Поддерживаются следующие типы команд:

- `status` - получение текущего состояния рабочей директории `git`.
- `diff` - получение различий между версиями или текущими изменениями.
- `commit` - сохранение новой версии кода агента.
- `rollback` - откат рабочей директории к выбранному стабильному снимку.
- `log` - получение списка стабильных снимков.

Для команды `diff` может передаваться `hash` коммита. Для команды `rollback` передается `target_sha` - ссылка на коммит, к которому нужно выполнить откат. Для команды `commit` передается текстовое сообщение `message`, описывающее внесенные изменения.

12.2 GitRequest

`GitRequest` - это общий контракт запроса к наблюдателю для выполнения `git`-команды. Он содержит тип сообщения `git` и вложенную команду `command`. Благодаря использованию дискриминатора `command` система может автоматически определить конкретный тип команды и проверить корректность ее параметров.

Использование таких контрактов снижает вероятность передачи некорректных сообщений между агентом, клиентом и наблюдателем, а также делает протокол взаимодействия более формальным и расширяемым.

13 Наблюдатель

Наблюдатель - это модуль, который разворачивается внутри контейнера supervisor и отвечает за управление жизненным циклом агента, контроль версий кода через Git, выполнение отката агента к стабильной версии при ошибках, а также координацию взаимодействия между клиентом и агентом через брокер сообщений RabbitMQ.

Основные функции:

- Инициализация и управление Git-репозиторием агента.
- Мониторинг состояния агента.
- Обработка команд управления версиями: commit, rollback, diff, status, log.
- Автоматический перезапуск агента при обновлении кода.
- Восстановление работоспособной версии агента при возникновении ошибок.
- Обработка запросов клиента к данным чатов.

Наблюдатель состоит из следующих подмодулей:

1. RabbitMQSupervisor - прием и отправка сообщений через очереди RabbitMQ.
2. GitService - управление Git-репозиторием, вызываемые функции для выполнения commit, rollback, diff, status и log.
3. Message Handlers - обработчики команд от агента и клиента.
4. Rollback - перезапуск контейнера агента через Docker SDK.

13.1 Класс RabbitMQSupervisor

Данный класс наследуется от RabbitMQBase. Помимо наследуемых методов для подписки на очередь и начала ее прослушивания содержит следующие методы:

- Метод `send_start_command`. Отправляет команду запуска агенту с необходимыми параметрами контекста.
Параметры: `task` - текст задачи от пользователя, `error_text` - текст ошибки для обработки при восстановлении после сбоя, `agent_session` - данные текущей сессии агента, `replayed_task` - признак повторного запуска задачи после перезапуска или отката.
При вызове формирует сообщение с командой `start`, добавляет в сообщение переданные параметры и публикует его в очередь агента через `publish_message()`.
- Метод `send_ready_message`. Уведомляет клиента о готовности системы к работе. Формирует сообщение с типом `ready` и командой `start`, после чего отправляет сообщение в очередь клиента через `publish_message()`.
- Метод `receive_message`. Обработчик сообщений с командой `start` от клиента и сообщений с типами `client_data`, `git`, `error`, `ack` или `login`. При получении `start` вызывает метод `send_start_command` для запуска агента. При получении сообщений от агента вызывает соответствующие обработчики. В случае `ack` дополнительно отправляет сообщение клиенту о готовности агента к новым запросам через `send_ready_message()`.

13.2 Обработчики сообщений

Функция `error_handler` - обработка сообщений об ошибках агента. Основная задача данной функции - вернуть агента в рабочее состояние. Для этого выполняется поиск последней стабильной версии в базе данных, регистрируется ошибка, выполняется откат репозитория к стабильной версии и перезапускается агент.

Функция `ack_handler` - обработка подтверждения успешного выполнения задачи агентом. Изменяет статус последней версии агента в базе на `STABLE` и выполняет проверочный автокоммит на случай, если код в рабочей директории агента отличается от кода в последнем коммите `git`-репозитория.

Функция `git_handler` - обработка `git`-запросов. В зависимости от команды вызывает `status`, `diff`, `commit`, `rollback` или `log`. Перед выполнением `commit` дополнительно проверяет синтаксис `Python`-файлов. Если обнаружена ошибка компиляции, коммит не создается, а вызывающему компоненту возвращается текст ошибки.

Функция `client_data_handler` - обработка запросов клиента к данным чатов. Поддерживает действия `list_chats`, `create_chat`, `rename_chat`, `get_history` и `add_client_message`.

13.3 GitService

`GitService` - это подмодуль наблюдателя, отвечающий за управление `git`-репозиторием рабочей директории агента. Он используется для фиксации изменений, получения различий между версиями, отката к стабильному состоянию и хранения истории стабильных снимков.

13.3.1 Назначение

Основная задача `GitService` - обеспечить безопасное изменение кода агента. Каждое значимое изменение фиксируется в `git`-репозитории и связывается с записью в базе данных. Если после изменения возникает ошибка, наблюдатель может найти последнюю стабильную версию и восстановить рабочую директорию.

13.3.2 Основные функции

- Инициализация `git`-репозитория при первом запуске.
- Проверка существования рабочей ветки `main`.
- Создание начального стабильного коммита.
- Выполнение автоматического коммита при наличии несохраненных изменений.
- Проверка синтаксиса `Python`-файлов перед сохранением новой версии.
- Получение статуса и `diff` рабочей директории.
- Откат рабочей директории к выбранному коммиту.
- Получение списка стабильных снимков из базы данных.

13.3.3 Логика сохранения версии

Перед сохранением новой версии GitService выполняет компиляцию всех Python-файлов рабочей директории. Если обнаружена синтаксическая ошибка, коммит не создается, а вызывающему компоненту возвращается текст ошибки с указанием файла и позиции.

Если ошибок не найдено, GitService добавляет изменения в индекс git и создает коммит с переданным сообщением. После успешного коммита в базе данных создается запись Snapshot со статусом PENDING. Такой статус означает, что версия создана, но еще не подтверждена как стабильная.

После успешного выполнения задачи агент отправляет подтверждение ask. Наблюдатель переводит последний снимок со статусом PENDING в статус STABLE. Если в процессе работы возникает ошибка, снимок переводится в статус ERROR, а система выполняет откат к последней стабильной версии.

13.4 Rollback

Модуль rollback отвечает за перезапуск контейнера агента. Имя контейнера берется из переменной окружения AGENT_CONTAINER_NAME. Если контейнер находится в состоянии exited, он запускается. Если контейнер уже работает, он перезапускается.

13.5 Логика работы наблюдателя

При первом запуске контейнера наблюдатель выполняет инициализацию git-репозитория в именованном томе agent_git при его отсутствии. Если репозиторий уже существует, наблюдатель выполняет автокоммит, если код в рабочей директории агента отличается от кода в последнем коммите git-репозитория.

Далее наблюдатель отправляет в очередь клиента сообщение о готовности агента и начинает слушать очередь supervisor_queue. В эту очередь могут приходить сообщения от агента или клиента. Клиент может отправить наблюдателю запрос, который нужно переслать агенту вместе с данными о контексте. Агент может отправить запрос на выполнение определенной git-команды или ответ о том, что агент выполнил текущую задачу и готов к новой, после чего будет выполнен автокоммит, а клиенту в очередь будет отправлено сообщение о готовности агента.

Также наблюдатель может получить от агента сообщение с типом error и задачей. В этом случае наблюдатель выполнит поиск последней стабильной версии в базе данных, откатит агента к этой версии и перезапустит его. После перезапуска наблюдатель отправит агенту сообщение с задачей, контекстом и текстом ошибки, чтобы агент мог продолжить выполнение запроса пользователя.

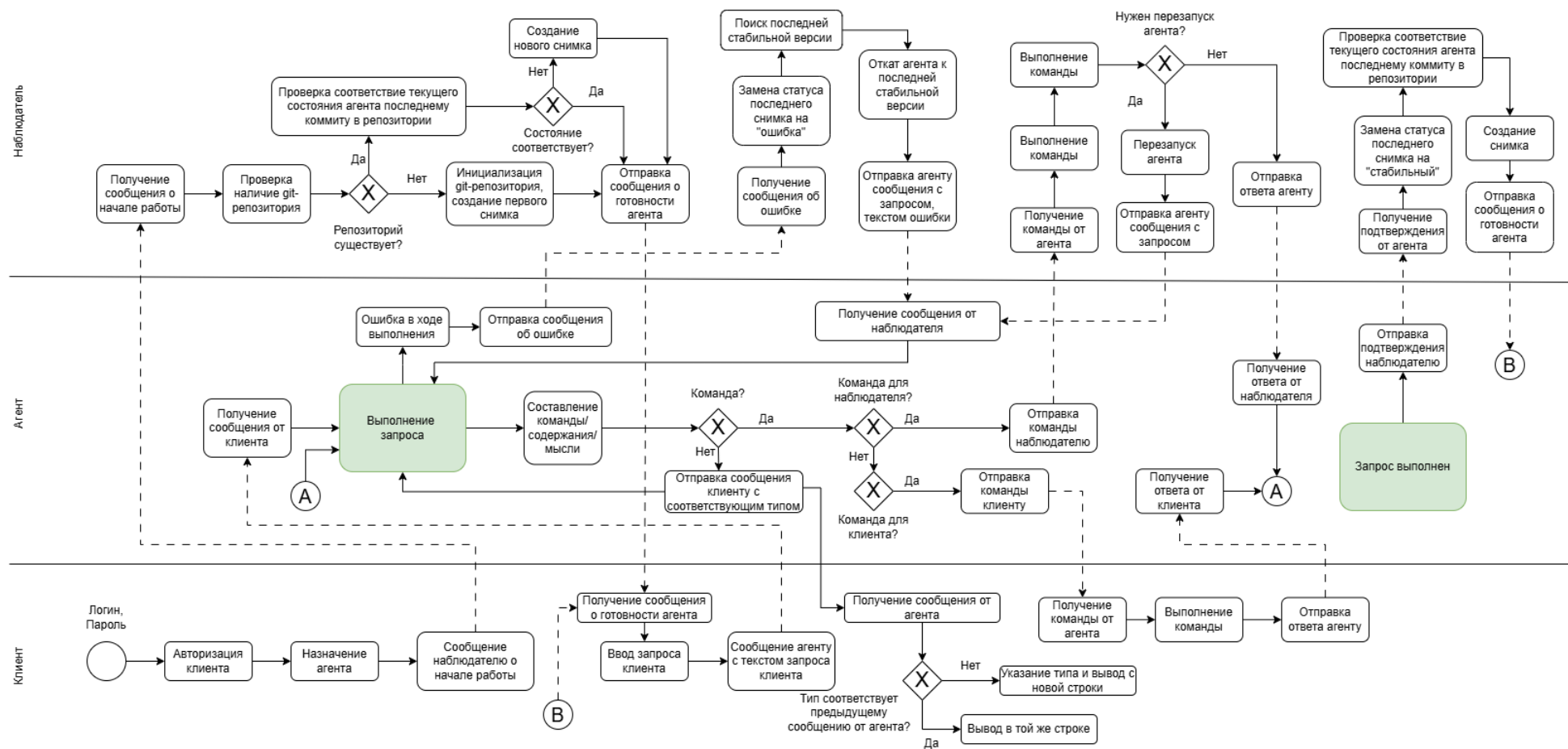


Рис. . Схема работы наблюдателя.

14 Маршрутизатор

Маршрутизатор - это центральный оркестратор мультиагентной системы. Он отвечает за аутентификацию пользователей, динамическое развертывание изолированных контейнеров для каждой пользовательской сессии и маршрутизацию сообщений между внешними клиентами и внутренними сервисами.

Основные функции:

- Аутентификация и авторизация пользователей.
- Динамическое управление инфраструктурой пользователей: создание, перезапуск и остановка контейнеров.
- Вычисление и распределение уникальных сетевых портов для каждого пользователя.
- Обеспечение изоляции данных между пользователями.
- Передача клиенту реквизитов подключения к персональному RabbitMQ.

Модули маршрутизатора:

- RabbitMQRouter - выполняет прием RPC-запросов на аутентификацию от клиентов, отправку ответов с реквизитами подключения.
- DockerManager - управляет жизненным циклом контейнеров агентов, выполняет docker compose команды для создания, запуска и остановки контейнеров.
- RouterDatabase - хранение учетных записей пользователей, хешей паролей и выделенных портов.

14.1 Класс DockerManager

DockerManager инкапсулирует всю логику взаимодействия с Docker через интерфейс командной строки. Отвечает за создание, перезапуск и остановку инфраструктуры пользователей.

Содержит следующие методы:

- Метод `start_client`. Создает новую инфраструктуру для пользователя при первом входе. Для этого метод вычисляет уникальные порты, вызывает `_run_compose()` с флагом `build=True` для сборки образов. Возвращает словарь с портами для сохранения в базу данных.
- Метод `restart_client`. Перезапускает существующую инфраструктуру пользователя при повторном входе. Вызывает `_run_compose()` с флагом `build=False`.
- Метод `_run_compose`. Внутренний метод выполнения команды `docker compose` для профиля `agent`. Метод формирует переменные окружения `LOGIN`, `DB_PORT`, `RABBITMQ_PORT` и `RABBITMQ_MGMT_PORT`, формирует команду и выполняет ее через `subprocess.run()`.
- Метод `stop_client`. Останавливает и удаляет инфраструктуру пользователя. При вызове метод формирует команду остановки и вызывает ее через `subprocess.run()`.

Порты пользователя вычисляются на основе значения `hash(login)`. Для базы данных используется смещение от 15432, для RabbitMQ - от 15672, для management-интерфейса RabbitMQ - от 25672. После первого входа выделенные порты сохраняются в базе данных маршрутизатора.

14.2 Класс RabbitMQRouter

RabbitMQRouter обеспечивает взаимодействие Router с брокером сообщений RabbitMQ. Реализует прием RPC-запросов от клиентов, аутентификацию и отправку ответов. В конструкторе класса устанавливается блокирующее соединение с RabbitMQ connection, настраивается канал связи channel для выполнения операций. Также инициализируется экземпляр класса DockerManager для управления контейнерами.

Класс содержит следующие методы:

- Метод `start_consuming`. Запуск бесконечного цикла ожидания и обработки входящих сообщений. Привязывает метод `receive_message` как callback-функцию к очереди роутера. Вызывает `channel.start_consuming()`, блокируя текущий поток.
- Метод `receive_message`. Обработка входящих сообщений. Метод маршрутизирует логику в зависимости от типа запроса: `login` или `logout`. В первом случае проверяет наличие данных пользователя в базе данных и соответствие пароля. Если пользователь новый, вызывает `start_client(login)`, иначе при совпадении пароля вызывает `restart_client(login)`.
- Метод `send_response`. Реализация механизма ответа в паттерне RPC. Публикует сообщение в обменник по умолчанию. В качестве ключа маршрутизации используется ключ на временную очередь, созданную клиентом специально для этого ответа.
- Метод `send_error`. Отправляет клиенту сообщение об ошибке, например при неверном логине или пароле.
- Метод `_schedule_logout`. Планирует остановку инфраструктуры пользователя с задержкой.
- Метод `_cancel_logout`. Отменяет запланированную остановку, если пользователь снова вошел в систему.

В ответ на успешный вход формируется сообщение `login_response`. Оно содержит внешний адрес RabbitMQ, порт персонального брокера, имя пользователя `client`, пароль подключения и логин пользователя. Внешний адрес берется из переменной окружения `SERVER_EXTERNAL_IP`, а при ее отсутствии используется `localhost`.

14.3 RouterDatabase

База данных маршрутизатора хранит таблицу `users`. В ней содержатся логин пользователя, хеш пароля, порт базы данных пользователя, порт RabbitMQ и порт management-интерфейса RabbitMQ. Пароль не хранится в открытом виде: при регистрации используется `bcrypt`-хеш.

14.4 Логика работы маршрутизатора

После запуска маршрутизатор начинает слушать соответствующую очередь RabbitMQ. При получении сообщения от клиента модуль пытается найти логин в базе данных. Если логин там есть, происходит проверка пароля, и при успехе запускается принадлежащая этому пользователю инфраструктура. Также из базы данных достаются данные о портах, необходимые для подключения клиента к персональному брокеру сообщений.

Если логина в базе нет, вычисляются порты для контейнеров, создаются новые контейнеры агента при помощи `docker compose` команды. Данные клиента помещаются в базу данных, а самому клиенту отправляется ответ с необходимыми реквизитами.

Когда маршрутизатору приходит сообщение с типом `logout` и логином пользователя, выполняется отложенная остановка соответствующих пользователю контейнеров. Если пользователь успевает снова войти до истечения задержки, остановка отменяется.

15 Протокол обмена сообщениями

Обмен сообщениями между компонентами системы строится поверх RabbitMQ. Для обычной публикации сообщений используется routing key, а для запросов, требующих ответа, применяется RPC-подход с временной очередью ответа и correlation_id.

15.1 Сообщения маршрутизатора

Маршрутизатор обрабатывает сообщения двух основных типов:

- login - вход пользователя в систему.
- logout - выход пользователя из системы.

При получении login маршрутизатор проверяет логин и пароль. Если пользователь отсутствует в базе данных, создается новая запись пользователя, вычисляются персональные порты и запускается его инфраструктура. Если пользователь уже существует, проверяется хеш пароля и перезапускается существующая инфраструктура.

В ответ на успешный login клиент получает сообщение login_response, содержащее адрес RabbitMQ, порт, имя пользователя, пароль и логин. Эти данные используются клиентом для подключения к персональной очереди пользователя.

При получении logout маршрутизатор не обязательно останавливает контейнеры сразу. Остановка может быть отложена на заданное количество секунд. Это позволяет избежать лишнего пересоздания инфраструктуры при кратковременном отключении клиента.

15.2 Сообщения наблюдателя

Наблюдатель обрабатывает следующие типы сообщений:

- start - запуск обработки задачи агентом.
- git - выполнение git-команды.
- error - сообщение об ошибке агента.
- ack - подтверждение успешного завершения задачи.
- login - проверка готовности агента после подключения клиента.
- client_data - запросы клиента к данным чатов.

При получении start наблюдатель пересылает агенту команду запуска вместе с задачей и данными текущей сессии. При получении git наблюдатель вызывает соответствующий обработчик GitService. При получении error выполняется регистрация ошибки, откат к стабильному снимку и перезапуск агента. При получении ack последняя ожидающая версия переводится в стабильное состояние.

16 Docker-инфраструктура

Для запуска системы используется Docker Compose. В конфигурации выделены два профиля: `router` и `agent`. Профиль `router` запускает центральную инфраструктуру маршрутизации, а профиль `agent` используется для запуска персональной инфраструктуры конкретного пользователя.

16.1 Профиль `router`

Профиль `router` включает следующие сервисы:

- `router_db` - база данных PostgreSQL для хранения учетных записей пользователей и выделенных портов.
- `router_rabbitmq` - брокер сообщений для входа клиентов и передачи `login/logout`-запросов.
- `router` - сервис маршрутизатора, который слушает очередь `router_queue` и управляет пользовательскими контейнерами.

16.2 Профиль `agent`

Профиль `agent` включает персональные сервисы пользователя:

- `agent_db` - база данных PostgreSQL для снимков, ошибок, истории чатов и памяти.
- `agent_rabbitmq` - брокер сообщений для взаимодействия клиента, наблюдателя и агента.
- `agent` - контейнер, в котором выполняется агент.
- `supervisor` - контейнер наблюдателя, управляющий `git`-состояниями и перезапуском агента.

16.3 Тома и изоляция

Для хранения состояния используются Docker volumes. Том `agent_work` содержит рабочую директорию агента, а том `agent_git` содержит `git`-репозиторий, через который наблюдатель управляет версиями. Для каждого пользователя Docker Compose запускается с отдельным `project name`, сформированным на основе логина. Это позволяет изолировать контейнеры и данные разных пользователей.

Контейнеры `router` и `supervisor` имеют доступ к `Docker socket`. Это необходимо для запуска, остановки и перезапуска пользовательских контейнеров изнутри сервисов управления.

17 Контроль качества и зависимости

Для контроля качества кода используется Ruff. Конфигурация расположена в файле `pyproject.toml`. В ней задана целевая версия Python 3.14, максимальная длина строки и набор правил статического анализа.

В проекте также используется GitHub Actions. При отправке изменений в основную ветку или при создании pull request выполняются следующие проверки:

- установка Python 3.14;
- установка зависимостей из `requirements/dev.txt`;
- проверка форматирования командой `ruff format --check`;
- статический анализ командой `ruff check`.

Зависимости проекта разделены по назначению:

- `requirements/agent.txt` - зависимости контейнера агента.
- `requirements/supervisor.txt` - зависимости наблюдателя.
- `requirements/dev.txt` - инструменты разработки и проверки качества.
- `router/requirements.txt` - зависимости маршрутизатора.
- `requirements/prod.txt` - объединенный набор зависимостей для production-сборки.

18 Клиент

Клиент – это локальная часть системы, которая запускается на машине пользователя. Клиент отвечает за вход в систему, отображение истории чатов, отправку задач агенту, получение сообщений модели и выполнение разрешённых команд на стороне пользователя. Именно через клиент агент получает доступ к рабочей директории пользователя, поэтому эта часть системы является важной границей между серверной инфраструктурой и локальной файловой системой.

В проекте реализованы два варианта клиента:

- графический клиент на PyQt6, запускаемый через `client/qt_main.py`;
- консольный клиент, запускаемый через `client/main.py`.

Основным вариантом является Qt-клиент. Он предоставляет экран входа, список чатов, окно переписки, настройки модели, настройки темы и выбор рабочей директории агента на машине пользователя.

18.1 Структура клиента

Клиентская часть разделена на три основных слоя:

- **UI-слой** – страницы и виджеты интерфейса в каталоге `client/ui/`;
- **контроллер** – модуль `ChatController`, связывающий интерфейс, состояние и сетевой клиент;
- **сетевой слой** – `RabbitMQClient`, отвечающий за обмен сообщениями с маршрутизатором, наблюдателем и агентом.

При запуске графического клиента создаётся объект `HyperagentClientWindow`. Внутри него инициализируются страницы входа, списка чатов, переписки и настроек. Состояние клиента хранится в `ClientState`: сохранённая сессия, текущий чат, настройки модели, тема интерфейса, положение окна и рабочая директория.

Если клиент запускается из контекстного меню `Windows` или из консоли с параметром `-work-dir`, этот путь сохраняется как рабочая директория агента. Если параметр не передан, рабочей директорией становится текущий каталог процесса. Это позволяет запускать `hyperagent-client.exe` из любой папки и автоматически привязывать новую задачу к этой папке.

18.2 Аутентификация и подключение

Первое подключение выполняется к центральному маршрутизатору. Клиент отправляет сообщение типа `login`, содержащее логин и пароль пользователя. Маршрутизатор проверяет данные, запускает или перезапускает персональную контейнер с агентом пользователя и возвращает ответ `login_response` с портами очередей агента.

После получения ответа клиент закрывает соединение с маршрутизатором и переподключается к персональному брокеру пользователя. Далее клиент отправляет наблюдателю сообщение `login`, чтобы проверить готовность агента. Когда наблюдатель присылает сообщение `ready`, интерфейс снимает блокировку и пользователь может отправлять задачи.

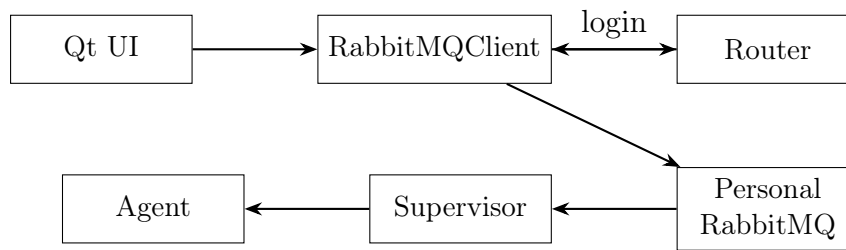


Рис. 8: Подключение клиента: сначала маршрутизатор, затем персональный RabbitMQ пользователя

18.3 Работа с чатами

Клиент не хранит историю переписки только локально. Список чатов, создание нового чата, переименование и загрузка истории выполняются через наблюдателя сообщениями типа `client_data`. Это позволяет хранить историю централизованно в базе данных пользователя и восстанавливать её после перезапуска клиента.

Перед отправкой пользовательского запроса клиент сохраняет сообщение пользователя как клиентское сообщение, затем отправляет наблюдателю команду запуска агента:

```
{ command: "start task: "... agent_session: {...} }
```

В `agent_session` передаются выбранный тип модели, параметры провайдера, уровень доступа к локальным командам и идентификатор текущего чата. Благодаря этому агент может продолжить работу в нужном чате и использовать настройки, выбранные пользователем в интерфейсе.

18.4 Рабочая директория пользователя

Рабочая директория – это каталог на машине пользователя, в котором выполняются клиентские команды агента. Она используется только для инструментов, которые действительно должны работать на стороне клиента: например, запуск PowerShell, Bash или Python-команд над локальными файлами пользователя.

Каталог может быть задан тремя способами:

- вручную в настройках клиента;
- параметром запуска `-work-dir`;
- автоматически как текущая директория консоли при запуске `hyperagent-client.exe`.

Для удобства Windows-сценария сборочный скрипт создаёт файл `hyperagent-client.exe`, добавляет папку `dist` в пользовательский `PATH` и регистрирует пункт контекстного меню. При запуске из контекстного меню выбранная директория передаётся в клиент как `-work-dir`, поэтому агент сразу работает с тем каталогом, из которого пользователь открыл приложение.

18.5 Выполнение клиентских команд

Когда агенту нужно выполнить действие на машине пользователя, наблюдатель пересылает клиенту сообщение типа `client_command`. Клиент создаёт рабочую директорию при необходимости, запускает команду через `subprocess.run()` с указанным `cwd`, собирает `stdout`, `stderr`, код возврата и отправляет результат обратно через RPC-ответ RabbitMQ.

Перед выполнением команды применяется пользовательский режим доступа:

- `read_only` – клиентские команды не выполняются;
- `ask` – перед каждой командой клиент показывает запрос на подтверждение;
- `full_access` – команды выполняются без дополнительного подтверждения.

В режиме подтверждения пользователь может разрешить одну команду, разрешить все команды в рамках текущего запроса или отклонить выполнение. Это позволяет агенту работать с локальной машиной пользователя, но сохраняет контроль над потенциально опасными действиями.

Заключение

Разработанная система решает поставленную задачу: агент способен выполнять пользовательские запросы, вызывать инструменты на стороне сервера и на стороне клиента и дополнять собственный код, оставаясь при этом под контролем инфраструктуры, которую он не может изменить. Разделение на mutable agent, immutable-обвязку, супервизора, маршрутизатор и хранилище данных даёт системе устойчивость к неудачным самомодификациям: любая новая версия проходит проверку синтаксиса и, при ошибке, откатывается к последнему стабильному снимку без участия человека.

Итеративное развитие архитектуры (V1–V4) последовательно снижало стоимость и время работы за счёт объединения ролей модели и слоистой памяти L0–L3, а версия V4 добавила накопление переиспользуемых навыков. Библиотека `llminterface` обеспечила эту эволюцию инфраструктурой, независимой от конкретного провайдера LLM и от тяжёлых внешних фреймворков, а чёткое разделение инструментов на серверные и клиентские устранило целый класс путаницы между файловой системой контейнера агента и машиной пользователя.

Эмпирическое сравнение 15 моделей на 165 прогонах показало, что при неизменной архитектуре агента выбор конкретной языковой модели даёт разброс среднего балла от 2.75 до 4.50 – то есть качество работы системы определяется не только архитектурой, но и моделью, на которой она работает, и это стоит учитывать при выборе модели по умолчанию и при дальнейшем развитии проекта.